Subject:	OOP-JAVA
Name:	Yug gandhi
ID No.:	25AIML013

Practical-13

GIT REPOSITRY LINK :

https://github.com/25aiml013-sudo/JAVA-PRACTICALS.git

Part A

PROBLEM-1 :

Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.

CODE :

```
class Box<T> {
    private T value;

    void set(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}
```

```
class Pair<A, B> {
    A first;
    B second;

    Pair(A first, B second) {
        this.first = first;
        this.second = second;
    }
}

public class GenericDemo {
    static <T> T first(T[] array) {
        return array[0];
    }

    public static void main(String[] args) {
        Box<Integer> b1 = new Box<>();
        b1.set(100);
        System.out.println(b1.get());

        Box<String> b2 = new Box<>();
        b2.set("Hello");
        System.out.println(b2.get());

        Pair<String, Integer> p = new Pair<>("Rahul", 20);
        System.out.println(p.first + " " + p.second);

        String[] names = {"Amit", "Rahul", "Priya"};
        System.out.println(first(names));
```

```
    }
}
```

OUTPUT :

```
(base) yuggandhi@Yugs-MacBook-Pro JAVA % cd "/Users/yuggandhi/Desktop/SEM-
ava GenericDemo
100
Hello
Rahul 20
Amit
(base) yuggandhi@Yugs-MacBook-Pro JAVA %
```

PROBLEM-2 :

Bounded stats: a generic method <N extends Number> double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

CODE :

```
public class AverageDemo {
    static <N extends Number> double average(N[] values) {
        double sum = 0;

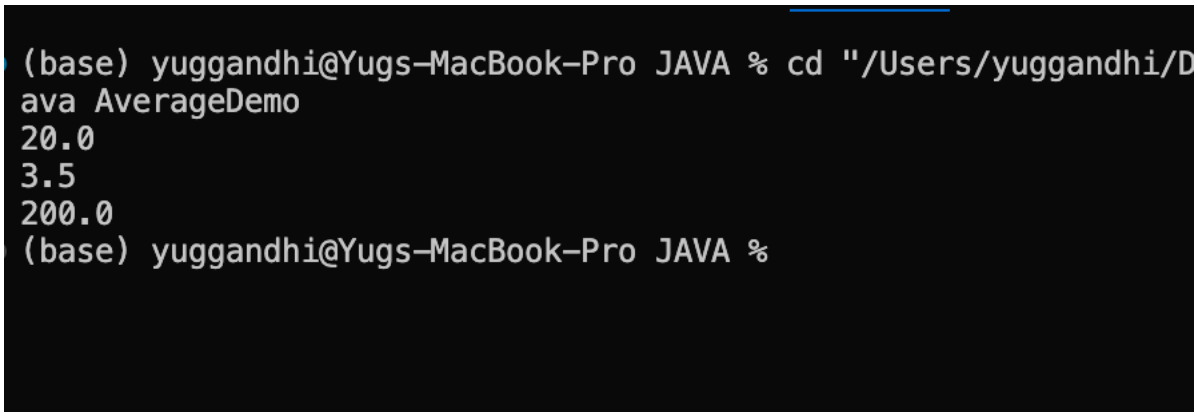
        for (N n : values) {
            sum += n.doubleValue();
        }

        return sum / values.length;
    }
```

```
    public static void main(String[] args) {
        Integer[] a = {10, 20, 30};
        Double[] b = {2.5, 3.5, 4.5};
        Long[] c = {100L, 200L, 300L};

        System.out.println(average(a));
        System.out.println(average(b));
        System.out.println(average(c));
    }
}
```

OUTPUT :



PROBLEM-3 :

Implement a small generic container of your choice (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty case sensibly.

CODE :

```
class Stack<T> {
    private Object[] data;
    private int top = -1;
```

```
    Stack(int size) {
        data = new Object[size];
    }

    void push(T value) {
        if (top == data.length - 1) {
            System.out.println("Stack is full");
            return;
        }
        data[++top] = value;
    }

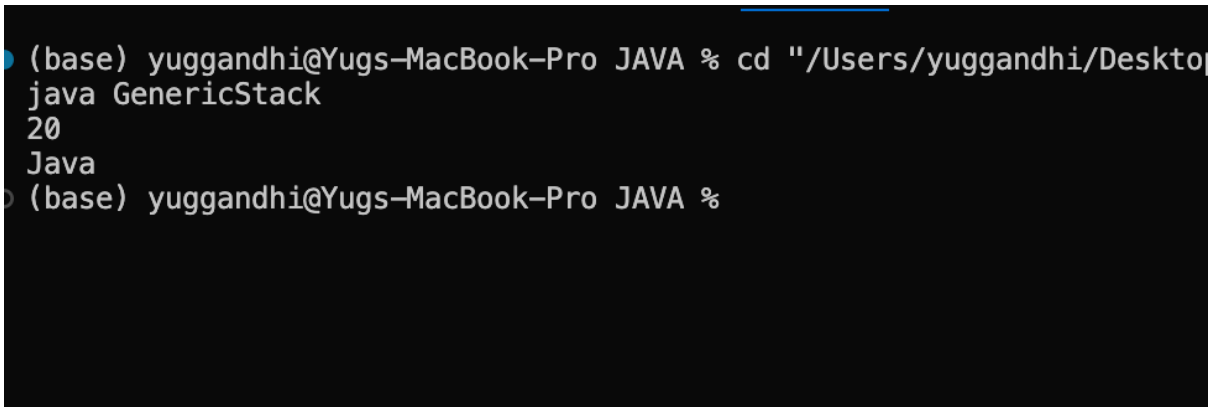
    @SuppressWarnings("unchecked")
    T pop() {
        if (top == -1) {
            System.out.println("Stack is empty");
            return null;
        }
        T value = (T) data[top];
        data[top--] = null;
        return value;
    }
}

public class GenericStack {
    public static void main(String[] args) {
        Stack<Integer> s1 = new Stack<>(5);
        s1.push(10);
```

```
        s1.push(20);
        System.out.println(s1.pop());

        Stack<String> s2 = new Stack<>(5);
        s2.push("Hello");
        s2.push("Java");
        System.out.println(s2.pop());
    }
}
```

OUTPUT :



Key Questions :

1. What problem do generics solve compared with using Object?
Generics provide type safety and reduce the need for type casting. They help detect type errors at compile time.

2. What is a bounded type parameter, and why is <N extends Number> useful?
A bounded type parameter restricts the types that can be used. <N extends Number> allows numeric types like Integer, Double, and Long to be used in calculations.

3. What is type erasure?

Type erasure is the process in which Java removes generic type information during compilation to maintain compatibility with older Java versions.